



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

The lecture defines a Table-Driven Agent as one that stores every possible percept sequence mapped to an action, guaranteeing perfect rationality by hardcoding the optimal move for every scenario. The problem is combinatorial explosion: if $|P|$ is the number of possible percepts and T is the agent's lifetime (number of steps), the table needs $|P|^T$ entries the size grows exponentially with how long the agent runs.

For Chess specifically, the slides cite roughly 10^{40} legal board states and an estimated 10^{120} possible game sequences (the Shannon Number). As the lecture puts it, there is no hard drive in the universe large enough to store that table, no human could write it, and no compiler could build it. And because the exponent is T (lifetime), the longer the agent operates, the table doesn't just grow it grows explosively; even a modest increase in T makes the table astronomically larger. This is exactly why the field moved from "store a table of answers" to "write an algorithm that

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
if percept['food_here']:
    action = 'move_forward'
elif percept['wall_ahead']:
    action = 'turn_left'
else:
    action = 'move_forward'
```

Each if/elif condition is checked directly against the current percept only — matching the lecture's pseudocode $\text{state} = \text{INTERPRET-INPUT}(\text{percept}) \rightarrow \text{rule} = \text{RULE-MATCH}(\text{state}, \text{rules}) \rightarrow \text{action} = \text{rule.ACTION}$. There is no `__init__`, no stored percept history, and no reference to anything beyond the immediate percept dictionary — this is exactly the "zero memory" property the slides emphasize, and it maps to the schematic diagram on slide 18: Sensors $\rightarrow$ "What the world is like now" $\rightarrow$ Condition-Action rules $\rightarrow$ Actuators, with no internal state block in between.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

It only sees the wall directly in front of it (partial observability) and remembers nothing from earlier steps (no percept history). In a corner, `wall_ahead` $\rightarrow$ `turn_left` eventually points it back at the same wall, and since it can't recall trying that already, it repeats the same turn forever.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Transition Model — `_update_state()` predicts how facing/position change based on the last action taken (turning or moving), just like predicting how the world evolves from an action.

Sensor Model — that prediction is checked against the live percept each turn, so belief and reality stay in sync.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING